CSC2042S Assignment 1 Perceptron Image Classification

Student number RHMABD006

This report explains the design testing and results of a multiclass perceptron used to classify images from the Simpsons MNIST dataset. The dataset has 28 by 28 images of 10 Simpsons characters. These characters are Bart Burns Homer Krusty Lisa Marge Milhouse Moe Ned and Skinner. The dataset has both RGB and grayscale images. There are 8000 training images and 2000 test images for each type. The classifier was made from scratch using only NumPy. It uses a one vs rest method with 10 separate binary perceptrons. Each perceptron is responsible for one Simpsons character.

Data Processing

The images were loaded using Pillow and changed into NumPy arrays. The RGB images have the shape 28 by 28 by 3 while the grayscale images have the shape 28 by 28.The 8000 training images for each type were split into 80 percent for training and 20 percent for validation. This gave 6400 training images and 1600 validation images. The split was done by class so that each of the 10 characters had about the same number of images in each group. The 2000 test images were not used during training or tuning. They were only used for the final testing. The images were changed into one long list of numbers before being given to the perceptron. RGB images had 2352 features because 28 times 28 times 3 equals 2352. Grayscale images had 784 features because 28 times 28 equals 784. Three different ways of normalising the data were tested. These were no normalisation min max scaling from 0 to 1 and z score standardisation. For z score standardisation the values were calculated using only the training data. The same values were then used for the validation and test data. This was done to prevent data leakage.

Model Design

The classifier uses the one vs rest method as required.A BinaryPerceptron predicts whether an image belongs to one class or not. It gives a value of 1 if the image belongs to that class and 0 if it does not. The weights and bias were both started at 0.1. When the perceptron makes a wrong prediction the weights are updated using the perceptron learning rule. The MultiClassPerceptron contains exactly 10 BinaryPerceptrons. Each one represents one of the 10 Simpsons characters. During training each BinaryPerceptron is trained separately. The labels are changed so that images belonging to that character have a label of 1 and all other images have a label of 0. All 10 perceptrons use the same shuffled order of training examples during each epoch. When making a prediction the model looks at the raw score from each of the 10 perceptrons. It then chooses the character with the highest score.

The model does not use the final 0 or 1 values from each perceptron when choosing the class. This is important because using only the 0 or 1 values could cause problems if no perceptron or more than one perceptron gives a positive result.

Training and Stopping Criteria

Two different ways of deciding when training should stop were tested. The first method used a fixed number of epochs. The model simply trained for a set number of epochs. The second method used early stopping. With early stopping the model stops training when the validation accuracy does not improve for a certain number of epochs. This number is called patience. The training data was shuffled after every epoch for both methods. For the RGB data the fixed epoch method did not show a smooth improvement. The validation accuracy could change by about 10 to 15 percentage points between two epochs even after 10 epochs. Early stopping usually stopped at a similar point to the fixed epoch method. Sometimes it stopped slightly later because the validation accuracy could go up and down instead of always improving. This happens because the perceptron is sensitive to data that cannot be separated easily. Each of the 10 binary perceptrons has a difficult problem to solve. Because of this the validation accuracy can change a lot between epochs.

Hyperparameter Tuning

A grid search was used to find good settings for the learning rate and normalisation method. Three learning rates were tested. These were 0.01 0.1 and 1.0. Three normalisation methods were also tested. These were no normalisation min max scaling and z score standardisation. The RGB and grayscale data were tested separately. Only the training and validation data were used during this process. The test data was not used.

RGB results

Normalisation	**LR 0.01**	**LR 0.1**	**LR 1.0**
None	23.19%	25.81%	25.75%
Min max	30.00%	34.31%	25.44%
Z score	32.94%	31.19%	35.38%

The best RGB settings were a learning rate of 1.0 and z score normalisation. This gave a validation accuracy of 35.38%.

Grayscale results

Normalisation	**LR 0.01**	**LR 0.1**	**LR 1.0**
None	15.88%	12.44%	11.38%
Min max	16.19%	15.75%	15.00%
Z score	20.75%	19.75%	19.19%

The best grayscale settings were a learning rate of 0.01 and z score normalisation. This gave a validation accuracy of 20.75%. Z score normalisation was the best or close to the best method for both RGB and grayscale data. This suggests that changing the pixel values to have a similar scale helps the perceptron learn better than using the original values or min max scaling. There was no single best learning rate for both types of images. RGB worked best with a higher learning rate of 1.0 while grayscale worked best with a lower learning rate of 0.01. This could be because the two types of images have different features and are easier or harder to separate after normalisation.

Evaluation

The best settings found during hyperparameter tuning were used to train the final models. Early stopping was used with a patience of 3 epochs. The final models were tested using both the validation data and the test data.

Results	**RGB**	**Grayscale**
Validation accuracy	36.75%	18.5%
Test accuracy	38.35%	18.05%

The RGB model had different results for different characters. The F1 scores ranged from 0.247 for Bart to 0.635 for Marge. The grayscale model had lower and more similar F1 scores. They ranged from 0.136 to 0.236. The confusion matrices showed that some RGB mistakes happened between characters with similar colours. For example Bart and Lisa both have yellow skin which can make them harder to tell apart. The grayscale model had more confusion between many different characters. This makes sense because grayscale removes colour which is an important feature that can help tell the characters apart.

RGB vs Grayscale Analysis

The RGB model performed much better than the grayscale model on the test data. The RGB model had a test accuracy of 38.35% while the grayscale model had a test accuracy of 18.05%. This means that RGB was about 20.3 percentage points better than grayscale. The results for each class show that colour helped the model recognise some characters. Marge had the best RGB F1 score at 0.635. One reason for this could be her blue hair which gives the model a strong colour feature to use. This colour information is lost when the image is changed to grayscale. Bart had the lowest RGB F1 score at 0.247. For grayscale Milhouse had the highest F1 score at 0.236 while Ned had the lowest at 0.136. The large difference between RGB and grayscale shows that colour is useful for this task when using a linear perceptron. Some characters are easier to separate when the model can use colour information. The grayscale model had much more difficulty.

Even though RGB performed better the accuracy was still fairly low. This shows that a simple linear perceptron is limited when trying to classify images. A linear model cannot create very complicated decision boundaries.

Data Augmentation Analysis

The best RGB settings from the hyperparameter tuning were used for the augmentation tests. These settings were a learning rate of 1.0 and z score normalisation. Three different conditions were tested. The first condition used no augmentation. The second condition used brightness changes. The third condition used small random rotations. Five models were trained for each condition using different random seeds. The augmented training sets increased from 6400 images to 12800 images. The average test accuracy and standard deviation were calculated from the five models for each condition.

Condition	Mean test accuracy	Standard deviation
None	**36.34%**	**1.52%**
Brightness	**36.85%**	**0.83%**
Rotation	**35.24%**	**0.54%**

Brightness augmentation gave a small improvement of 0.51 percentage points. However this improvement is smaller than the normal variation between the different runs of the model. The standard deviation for the model without augmentation was 1.52%. Because of this it is not clear that brightness augmentation actually improved the model. The small improvement could just be caused by random differences between runs. Rotation augmentation made the accuracy worse by 1.10 percentage points. This change was larger than the standard deviation for the rotation tests. This suggests that rotation probably had a real but small negative effect. One possible reason is that brightness changes keep the position of the pixels the same. It only changes how bright the pixels are. This means the model can still use the same colour and shape information. Rotation is different because it moves the pixels to different positions. The perceptron gives each pixel position its own weight. This means that moving the image can make the rotated image look very different to the model even though it still shows the same character. A more advanced model such as a convolutional neural network can deal with changes in position much better. A simple perceptron cannot do this as well. These results show that simply increasing the amount of training data does not always improve a model. The type of extra data matters. An augmentation method needs to work well with the type of model being used.

AI Usage

I used Claude during this assignment to help me understand the assignment requirements and clarify concepts that I did not fully understand. I also used Claude to help me navigate my terminal and Git, and to check my code for possible logical errors and help me identify areas that needed to be corrected.

I uploaded the assignment Agents file to Claude so that the AI could follow the assignment requirements and constraints when assisting me. I used the AI mainly as a support and debugging tool rather than as a replacement for my own work.

The core code for the perceptron implementation, training, evaluation and experiments was written by me. I ran the experiments using the actual assignment dataset and used the results produced by my own code for the report. I checked and corrected suggestions from the AI by testing the code myself and verifying that the results were consistent with the assignment requirements.

I also used AI to help explain errors and guide me when working with Git and the terminal. I remained responsible for the final code, experiments, results and conclusions submitted in this assignment.

Conclusion

The best RGB settings were a learning rate of 1.0 and z score normalisation. These settings gave a validation accuracy of 35.38%.

The best grayscale settings were a learning rate of 0.01 and z score normalisation. These settings gave a validation accuracy of 20.75%. The RGB model performed much better than the grayscale model. The RGB test accuracy was 38.35% while the grayscale test accuracy was 18.05%. This is a difference of about 20.3 percentage points. The results show that colour was very useful for this task. Characters with strong colour features such as Marge were easier for the model to recognise.cBrightness augmentation gave a small improvement, but the improvement was small enough that it could have been caused by normal differences between runs. Rotation augmentation made the model perform slightly worse. This was probably because rotating the images changes the position of the pixels and the perceptron depends heavily on fixed pixel positions. The one vs rest perceptron was able to learn useful information from the images. RGB images worked much better than grayscale images because the model could use colour information. However the overall accuracy was still fairly low. This shows the limits of using a simple linear perceptron for image classification. The low results also show that adding more training images does not automatically make the model better. The type of augmentation needs to match the type of model being used.